

Competitive Programming

Section 24: Grid DP

Competitive Programming Course

2026-07-06

Table of contents

1	Why grid DP is its own pattern	3
2	Recognition	4
3	Basic state	5
4	Counting paths with obstacles	6
5	Minimum-cost paths	7
6	Row compression	8
7	Adding direction to the state	9
8	First move convention	10
9	Local exercises	11
9.1	A. Safe Paths	11
9.2	B. Lowest Toll	11
9.3	C. Limited Turns	11
10	C++ implementation notes	12
11	Python implementation notes	13
12	Proof habits	14
13	Common mistakes	15
14	Local judge workflow	16
15	Submission checklist	17

1 Why grid DP is its own pattern

Grid problems often look like graph problems. A cell is a state, and movement between neighboring cells is an edge. But if movement is only right/down, or otherwise acyclic, dynamic programming is usually simpler than BFS or Dijkstra.

The main observation:

```
To reach cell (r,c) using only right/down moves,  
the previous cell is either (r-1,c) or (r,c-1).
```

That gives a natural computation order:

```
top row to bottom row  
inside each row, left to right
```

2 Recognition

Use grid DP when:

Statement signal	Likely state
move only right/down	<code>dp[r][c]</code>
count paths with obstacles	number of ways to reach each cell
minimize cost along right/down paths	minimum cost to reach each cell
maximize collected value along right/down paths	maximum value to reach each cell
path condition depends on last move	add direction to the state
path condition depends on number of turns	add turn count to the state

Use graph shortest path instead when moves can form cycles and have arbitrary costs. For example, moving up/down/left/right with positive tolls is Dijkstra, not this section's simple grid DP.

3 Basic state

For a right/down grid:

```
dp[r][c] = answer for paths from the start to cell (r,c)
```

The meaning of “answer” depends on the problem:

```
counting paths:  number of ways  
minimum toll:   minimum cost  
maximum score:  maximum score
```

The recurrence uses the incoming neighbors:

```
from above: (r-1,c)  
from left:  (r,c-1)
```

4 Counting paths with obstacles

If a cell is blocked, no path can stand there:

```
dp[r][c] = 0
```

Otherwise:

```
dp[r][c] = dp[r-1][c] + dp[r][c-1]
```

with missing neighbors ignored at the top and left borders.

Example:

```
....  
.#..  
....
```

Table of path counts:

```
1 1 1 1  
1 # 1 2  
1 1 2 4
```

So the answer is 4.

5 Minimum-cost paths

For tolls:

```
dp[r][c] = minimum cost to reach (r,c), including this cell's toll
```

Transition:

```
dp[r][c] = toll[r][c] + min(dp[r-1][c], dp[r][c-1])
```

At the start:

```
dp[0][0] = toll[0][0]
```

Use a large INF for missing neighbors if that makes the implementation cleaner.

6 Row compression

For basic right/down grid DP, each row depends only on:

```
previous row, same column  
current row, previous column
```

So for a minimum-cost grid, you can store only:

```
prev[c] = dp[r-1][c]  
cur[c]  = dp[r][c]
```

This reduces memory from $O(nm)$ to $O(m)$.

Use full tables while learning or when you need to reconstruct a path. Compress only after the recurrence is stable.

7 Adding direction to the state

Sometimes `dp[r][c]` is not enough. If the statement asks about turns, the future needs to know the last move direction.

For Limited Turns:

```
dp[r][c][t][dir]
```

means:

```
number of paths that end at (r,c),  
have used exactly t turns,  
and whose last move direction is dir
```

Directions:

```
RIGHT = 0  
DOWN  = 1
```

To enter (r, c) from the left, the new last direction is `RIGHT`.

```
continue right:  
    dp[r][c][t][RIGHT] += dp[r][c-1][t][RIGHT]  
  
turn from down to right:  
    dp[r][c][t][RIGHT] += dp[r][c-1][t-1][DOWN]
```

To enter from above, symmetric:

```
continue down:  
    dp[r][c][t][DOWN] += dp[r-1][c][t][DOWN]  
  
turn from right to down:  
    dp[r][c][t][DOWN] += dp[r-1][c][t-1][RIGHT]
```

8 First move convention

The first move does not count as a turn because there is no previous direction.

A clean implementation initializes the two possible first moves:

```
if cell (0,1) is open:
    dp[0][1][0][RIGHT] = 1

if cell (1,0) is open:
    dp[1][0][0][DOWN] = 1
```

Then every later change of direction is counted uniformly.

Special case:

```
n = 1 and m = 1
```

The empty path is valid and uses zero turns, so the answer is 1 if the single cell is open.

9 Local exercises

9.1 A. Safe Paths

Count right/down paths through open cells.

State:

```
dp[r][c] = number of ways to reach (r,c)
```

Operation:

```
sum incoming ways modulo 1_000_000_007
```

9.2 B. Lowest Toll

Find the minimum toll of a right/down path.

State:

```
dp[r][c] = minimum total toll to reach (r,c)
```

Operation:

```
cell toll + min(incoming costs)
```

This exercise uses row compression.

9.3 C. Limited Turns

Count right/down paths using at most K turns.

State:

```
dp[r][c][turns][last_direction]
```

This trains adding exactly the extra information the future needs.

10 C++ implementation notes

For counts:

```
const long long MOD = 1000000007LL;
```

For large costs:

```
const long long INF = (1LL << 62);
```

For four-dimensional DP, keep the dimensions modest and name direction constants:

```
const int RIGHT = 0, DOWN = 1;
```

11 Python implementation notes

For grid strings:

```
grid = [input().strip() for _ in range(n)]
```

For row-compressed minimum cost:

```
prev = [inf] * m  
cur = [inf] * m
```

For multi-dimensional DP, clarity matters more than cleverness. The nested list in Limited Turns is acceptable because the constraints are intentionally small.

12 Proof habits

Grid DP proofs are usually induction over row-major order.

Template:

```
Assume every cell before (r,c) in row-major order has correct DP values.  
Every valid path into (r,c) must come from an allowed incoming neighbor.  
The transition combines exactly those possibilities.  
Therefore dp[r][c] is correct.
```

For direction-aware states, prove the state is precise:

```
Every path ending with RIGHT is counted in the RIGHT state.  
Every path ending with DOWN is counted in the DOWN state.  
Changing between these states increments turns exactly once.
```

13 Common mistakes

```
Counting paths through blocked cells.  
Forgetting to initialize the start cell.  
Adding top and left neighbors before checking borders.  
Using BFS for acyclic right/down counting DP.  
Using int for large minimum costs.  
For Limited Turns, counting the first move as a turn.  
For Limited Turns, using dp[r][c][turns] without last direction.
```

14 Local judge workflow

Run this section:

```
CP_TARGET=solution python3 sections/24_grid_dp/check.py
```

Run everything:

```
CP_TARGET=solution python3 tools/check_all.py
```

15 Submission checklist

Before submitting:

```
What exactly does dp[r][c] mean?  
Are movement directions acyclic?  
Are blocked cells handled before transitions?  
Are borders initialized safely?  
Do I need modulo or long long?  
Does the statement require extra state such as direction, turns, or remaining budget?
```